

Tilemaps with Tiled

Overview

Chapter 10 built tilemaps from arrays in code. This chapter hands the level over to a tool: students draw maps in **Tiled**, export them as JSON, and load them into Phaser. The Phaser side is small - `tilemapTiledJSON`, `make.tilemap({ key })`, `addTilesetImage`, `createLayer`, `setCollisionByProperty`, and three ways into an object layer - so most of the learning is about the **workflow** (source file, export, build, refresh) and about the idea that a **level is data**: the code says what things do, the map says where they are and how they are tuned.

The one conceptual trap is **names as strings shared between two programs**. Tiled and the code must agree on the tileset name, every layer name, every object name and every property name, and a mismatch is not a build error - it is a console warning and a `null`. Teach students to keep the names as constants and to look in the browser console first.

Tiled is not on every lab machine. It is a free download (mapeditor.org) for Windows, macOS and Linux; check it is installed, and that it opens the chapter's maps, before the session. Without it, students can still do the Phaser half of the chapter with the maps provided, and challenges 2 and 4 can be done by editing the `.tmj` by hand (instructive in itself).

Prerequisites

- Chapter 10: tilemaps from arrays, tilesets, layers, `setCollision`, colliders with a layer, the camera following the player
- Chapters 7-9 for the hero animation and Arcade Physics used by the level (the chapter does not re-teach them)
- Chapter 2's `scene.start(key, data)` / `init(data)`, used again as `scene.restart(data)`

Learning outcomes

Students can:

1. make an orthogonal, fixed-size map in Tiled with an embedded tileset, tile layers, an object layer and custom properties on tiles, objects and the map
2. save the source (`.tmx`) and export the JSON (`.tmj`) to the right place, and explain what a GID and a firstgid are
3. load a Tiled map in Phaser and create its layers, matching the names used in Tiled
4. make tiles solid with `setCollisionByProperty` , and read any tile's custom properties
5. read an object layer with `findObject` , `filterObjects` , `createFromObjects` and `getObjectLayer(...).objects` , converting Tiled's rectangle positions to centres where needed
6. read custom properties on objects and maps, and explain why the code should cope with missing or wrongly typed values
7. join maps together with door objects whose properties name the target map and entrance

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Play <code>ch11_tiled_level1</code> . Ask: "where is this level in the code?" (It is not.) Open <code>level11.tmx</code> in Tiled on the projector
0:05 - 0:25	Live demo in Tiled (slides 3-8): New Map, New Tileset (embedded!), two tile layers, paint a few tiles, add <code>collides</code> to the ground tiles, an object layer with a <code>player</code> point and a couple of <code>coin</code> points, a map property. Save As in <code>tiled/</code> , Export As into <code>public/assets/maps/</code>
0:25 - 0:35	Slides 9-10: open the <code>.tmj</code> in Celbridge. GIDs and firstgid - ask "why does the first tile get GID 1, not 0?"
0:35 - 0:50	Slides 11-15: the four steps in <code>createMap()</code> , names that must match, <code>setCollisionByProperty</code> , press D. The three ways into an object layer; <code>getTiledProperty</code>
0:50 - 1:00	Slides 16-18: doors between maps, <code>scene.restart(data)</code> , loading a map on demand

Lab (2 hours)

Time	Activity
0:00 - 0:15	Install/start Tiled; open both projects' maps; build and run both projects
0:15 - 0:45	Guided: change <code>level11</code> in Tiled (move the goal, add a platform and two coins, change a slime's speed), export, build, refresh. Everyone must get round the edit-export-build loop at least twice
0:45 - 1:15	Challenges 1-3
1:15 - 2:00	Challenges 4-6

Key points to stress

- **Two files.** The game loads the `.tmj`, never the `.tmx`. "I changed the map and nothing happened" is nearly always a missing export (or a missing build)
- **Embed the tileset, fixed size, CSV.** Phaser cannot read external tilesets, infinite maps are awkward, and compressed tile data is not supported

- **Names are the contract** between Tiled and the code - constants in one file, spelled exactly
 - **GIDs are not tile indexes.** In a Tiled map, `tile.index` is the GID (grass top is 1, not 0). Choosing tiles by property avoids the whole question
 - **Tile properties vs object properties.** Phaser turns tile properties into an object (`tile.properties.hazard`) but leaves object and map properties as Tiled's list of `{ name, type, value }` - hence `getTiledProperty`
 - **Rectangles are measured from the top-left**, game objects from their centre
 - **Data can be wrong.** The designer is a person: default values and checks (`?? 0` , the fallback in `getTiledProperty` , `null` checks) are part of reading a map
 - In Phaser 4, **Tiled's tile animations play by themselves** (Phaser 3 needed a plugin or hand-written code - students will find old tutorials that say otherwise)
-

Common problems and errors

These messages were produced by breaking copies of the chapter projects.

What students see	Cause	Fix
console: Tilemap has no tileset "platform". Its tilesets are ['platform_tiles'], then the project's own error No tileset "platform" in the map, or no picture "platform_tiles" loaded	first argument of addTilesetImage does not match the tileset's name in Tiled	copy the name from Tiled's Tilesets panel (the warning lists the real names)
console: Texture key "platform_tiles" not found, then the same No tileset... error	the tileset picture was never loaded with this.load.image, or with a different key	load it; the second argument of addTilesetImage is the key
console: Invalid Tilemap Layer ID: ground and Valid tilelayer names: ['Background', 'Ground'], then TypeError: Cannot read properties of null (reading 'setCollisionByProperty')	layer name spelled differently (here, a lower-case g)	match Tiled exactly - capitals matter
console: No map data found for key level1, every layer invalid; Network tab shows a 404	wrong path in tilemapTiledJSON, or the map was never exported into public/assets/maps/	export to the right folder, then build
console: Failed to process file: tilemapJSON "level1" and SyntaxError: Unexpected token '<', "<?xml vers"... is not valid JSON	loading the .tmx instead of the .tmj	export JSON; load the .tmj
console: External tilesets unsupported. Use Embed Tileset and re-export, then TypeError: Cannot read properties of undefined (reading '2')	"Embed in map" was not ticked when the tileset was made	Embed Tileset button in the Tilesets panel, or the "Embed tilesets" export option, then export again

What students see	Cause	Fix
the player falls through everything; no error	no <code>collides</code> property, a differently spelled one, or a string property <code>"true"</code> rather than a bool	press D; check the property's name and type in Tiled
changes made in Tiled do not appear	not exported, or not built, or the browser shows the old page	File > Export (Ctrl+E), build, refresh
a coin or door is half a tile out	a rectangle's <code>x, y</code> used as its centre	add half the width and height
build error: <code>Type 'TilemapLayer TilemapGPULayer' is not assignable to type 'TilemapLayer'.</code>	storing <code>createLayer(...)</code> in a <code>TilemapLayer</code> field without the cast	<code>as Phaser.Tilemaps.TilemapLayer</code> (we did not ask for a GPU layer)
build error: <code>'spawn' is possibly 'null'.</code>	using <code>findObject</code> 's result without checking it	check for <code>null</code> (the map might not have the object)
build error: <code>Type 'number undefined' is not assignable to type 'number'.</code>	<code>spawn.x</code> used directly - Phaser's type makes <code>x</code> optional	<code>spawn.x ?? 0</code>
the door sends the player straight back	the entrance point is inside the other map's door rectangle	move the entrance a tile or two into the room
every coin tile collected at once (challenge 6)	the overlap with a tile layer reports empty places too	check <code>tile.index !== -1</code>

Suggested demos and live-coding moments

- **The contract, broken.** Rename the Ground layer in Tiled to "ground", export, refresh: the console warning lists the valid names. Rename it back. Then rename the tileset: a different warning

- **Data, not code.** Change a slime's `speed` property to 300 in Tiled, export, refresh. Then type `"fast"` into it (change its type to string): the fallback in `getTiledProperty` keeps the game running. Ask what would happen without the `typeof` check
- **GIDs.** Add `console.log(this.ground.getTileAt(0, 17).index)` at the end of `createMap()` : it prints 1, not 0, for a grass tile. Ask why
- **Draw order.** In Tiled, paint a bush on the Background layer over one of the crates, and drag Background above Ground in the Layers panel. Export: the bush is still hidden behind the crate, because Phaser draws layers in the order the code calls `createLayer` . Swap the two `createLayer` lines and it appears in front. Which wins - Tiled's order, or the code's?

Discussion questions

- What belongs in the map, and what in the code? (Positions, sizes, speeds, messages, which tiles are solid - map. What a coin does when touched, how a slime patrols - code.) Where would you put a slime's *damage*? The level's music?
- Why is `collides` better than `setCollision([1, 2, 3, 4, 5, 6, 11])` ? What happens to each when a designer adds a new solid tile to the tileset?
- `RoomScene` has no list of rooms. What are the advantages? What can go wrong that a list would have caught at start-up? (A mistyped `target` is only found when someone walks through that door.)
- The maps in this chapter were written by a script, not drawn in Tiled. When might a game generate its maps rather than have them drawn? (Roguelikes do exactly that.)

Extension ideas

- a level select: several `levelN.tmx` files and a menu that starts `GameScene` with the map key
- a tile property `friction` (float) on ice tiles, read under the player's feet to change their drag
- use a Tiled **object template** (`.tx`) for coins - and find out what Phaser does with it (templates must be detached on export: Preferences > Export Options > Detach templates)

- tile objects (Insert Tile, T) instead of points for coins, so the designer sees coins in the editor; `createFromObjects` with `gid` handles them
 - a minimap of the Tiled level with a second camera (Chapter 10)
-

Assessment ideas

- Practical: "Add a second level to `ch11_tiled_level`, reached by touching the flag, with at least one new kind of object configured by a custom property." Mark the map (in Tiled) as well as the code
 - Code reading: show `createMap()` and a screenshot of Tiled's Layers and Tilesets panels with one name different; ask what the console will say and which line fails
 - Short answer: explain GID and firstgid, and why a map with two tilesets needs them
 - Short answer: why does `getTiledProperty` take a fallback, and why does it check `typeof`?
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`. The changed maps are written by `tools/make_solution_maps.ts`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every code change is marked with a `// CHALLENGE n` comment.

Every challenge in this chapter also changes a **map**, which students do in Tiled. JSON cannot hold comments, so the solutions' maps were written by a script, `tools/make_solution_maps.ts`, which builds the chapter's maps and changes them - each change marked `CHALLENGE n` there instead. The maps open in Tiled like any other, so you can show the class what was changed. Each solution's README says which maps changed.

When marking, ask for both files: the `.tmx` in `tiled/` **and** the exported `.tmj` in `public/assets/maps/`. A student whose game does not show their change has almost always forgotten to export.

1. Above the player

Project: `solutions/ch11_challenge_1_above_the_player` (from `ch11_tiled_doors`)

Goal: see that layers are drawn in order, that depth can override it, and that a layer with no collision is walked through.

In Tiled: Layer > New > Tile Layer, named **Above**, dragged above Walls in the Layers panel. The solution paints a stone beam across the hall between the pillars, and the tops of the four pillars. Nothing on this layer has `collides` set, and the scene does not ask for it anyway.

`src/scenes/RoomScene.ts`

```
const ABOVE_DEPTH = 10; // CHALLENGE 1 - higher than the player (0), so
drawn over them
```

```

this.createPlayer(map);
this.physics.add.collider(this.player, walls);

// CHALLENGE 1 - the "Above" layer, drawn over the player. No collision is
// set on it, so the
// player walks underneath. Not every room has one: getLayer gives null for
// a layer that is not
// in the map (createLayer would warn in the console, and give null too).
if (map.getLayer(ABOVE_LAYER) !== null) {
    map.createLayer(ABOVE_LAYER, tiles).setDepth(ABOVE_DEPTH);
}

```

Making the layer after the player would be enough on its own - game objects are drawn in the order they are added - but `setDepth` says what is meant and survives someone moving the line. No cast is needed here: `setDepth` exists on both kinds of layer `createLayer` can return.

Tested with the harness: the player walks up through row 8 (the beam) without stopping, and is hidden while under it; in the cellar, which has no Above layer, there is no console warning.

Look for:

- the layer made **after** the player, or given a depth - students who make it with the other layers find it drawn *under* the player and are puzzled
- the cellar still working: a student who calls `createLayer("Above", ...)` unconditionally gets *Invalid Tilemap Layer ID: Above* in the console for the cellar (harmless here, but a good moment to talk about checking). Adding an empty Above layer to every map is an equally good answer
- not setting collision on the Above layer

2. Gems

Project: `solutions/ch11_challenge_2_gems` (from `ch11_tiled_level`)

Goal: a custom property on an object, read through `createFromObjects` ' data.

In Tiled: select the coins to change (several can be selected at once with Select Objects), and add an **int** property **value** = 5. The solution changes the four coins above the brick wall.

src/scenes/GameScene.ts

```
// CHALLENGE 2 - createFromObjects has put each coin's custom properties
// into its data. A coin
// with a "value" is shown as a gem; the others spin. (The whole group can
// no longer be told to
// play the spin animation - it would turn the gems back into coins.)
for (const sprite of coinSprites) {
    const coin = sprite as Phaser.GameObjects.Sprite; // createFromObjects
    // made Sprites (its default)
    const value: number = coin.getData(VALUE) ?? 1;
    coin.setData(VALUE, value);
    if (value > 1) {
        coin.setTexture(GEM_KEY);
    } else {
        coin.play(COIN_SPIN);
    }
}
```

```
this.score = this.score + coin.getData(VALUE); // CHALLENGE 2
```

A coin with no property gets `undefined` from `getData`, so `?? 1` gives it the default - and `setData` stores it, so collecting is one line. `gem.png` is copied into `public/assets/images/` and loaded in `preload()`.

Tested with the harness: 12 coins have value 1 and 4 are gems with value 5; collecting a gem makes the score 5, then a coin makes it 6.

Look for:

- the property read from the sprite's **data** (`getData`), not from `obj.properties` - both work, but `createFromObjects` has already done the work. Students who loop over `getObjectLayer` and use `getTiledProperty` have a perfectly good answer too
- `this.coins.playAnimation(COIN_SPIN)` removed or replaced - left in, it turns every gem back into a spinning coin, a nice bug to discuss
- the score reset in `init()` (Chapter 2's trap: R restarts the scene)

- a common wrong turn: a **string** property `"5"` . `getData` then returns the string, and `score + "5"` makes `"05"` . The type in Tiled matters

3. A third room

Project: `solutions/ch11_challenge_3_a_third_room` (from `ch11_tiled_doors`)

Goal: understand that the level is data. The right answer changes **no code**.

In Tiled:

1. make a new map, `tiled/treasure.tmx` (the solution's is 17 x 9 tiles). The quickest way is to open the cellar, **File > Save As** `treasure.tmx` , and change the size (Map > Resize Map) and the tiles: the embedded tileset, its `collides` properties and the layers all come with it. Starting from File > New works too, but then the tileset must be made again - embedded, named **dungeon_tiles**, with `collides` ticked on the same tiles
2. layers **Floor**, **Walls** and **Objects**, spelled as in the other rooms; a map property **name** = `The Treasure Room`
3. in the new room: a point `from_hall` , and a `door` rectangle with `target` = `hall` , `entrance` = `from_treasure`
4. in the hall: repaint the closed door (tile 5) as an open door (tile 6), add a `door` rectangle over it with `target` = `treasure` and `entrance` = `from_hall` , and a point `from_treasure` two tiles below it
5. export both maps to `public/assets/maps/` - the new one's file name must be the `target` : `treasure.tmj`

`RoomScene` loads `assets/maps/<target>.tmj` on demand, finds the entrance by name and centres the camera on whatever size of room it gets - so nothing in `src/` changes.

Tested with the harness: walking up the carpet goes from the hall to the treasure room, arriving at `from_hall` ; walking down through its door returns to the hall at `from_treasure` .

Look for:

- no code changes. If a student changed code, ask what for - often they added the map to a list in `preload()` , which works but misses the point

- entrance points placed a tile or two **away** from the doors (otherwise the player arrives in the doorway and is sent straight back - students meet this and have to reason about it)
 - the three names that must agree across two files: the `target` and the new map's file name; the `entrance` and the point's name; the layer and tileset names
 - common failures: the file exported as `Treasure.tmj` with `target = treasure` (a 404 - file names on the web are case-sensitive), a non-embedded tileset (*External tilesets unsupported. Use Embed Tileset and re-export*), or forgetting to export the hall after changing it
-
-

4. Signposts

Project: `solutions/ch11_challenge_4_signposts` (from `ch11_tiled_level`)

Goal: trigger areas from rectangle objects, a string property, and asking "am I in it?" every frame.

In Tiled: rectangles named **sign**, each with a **string** property **message**. The solution has three (by the sign at the start, before the lava, and before the flag) and paints two more sign tiles in the Background layer. The first message has two lines; in the JSON a line break is written as `\n`, and Phaser's `Text` shows it as a new line.

`src/scenes/GameScene.ts`

```
// CHALLENGE 4 - a signpost: a rectangle from Tiled, and the message to show
while the player is in it
interface Sign {
  zone: Phaser.GameObjects.Zone;
  message: string;
}
```

```

for (const obj of signObjects) {
  const width = obj.width ?? 32;
  const height = obj.height ?? 32;
  // Tiled measures a rectangle from its top-left corner; a Zone is placed
  // by its centre
  const zone = this.add.zone((obj.x ?? 0) + width / 2, (obj.y ?? 0) + height
  / 2, width, height);
  this.physics.add.existing(zone, true);
  this.signs.push({ zone: zone, message: getTiledProperty(obj.properties,
  "message", "") });
}

```

```

// CHALLENGE 4 - show the message of the sign the player is standing by, or
// hide the box.
// physics.overlap with no callback just answers the question: are these two
// touching right now?
private updateSignMessage(): void {
  const sign = this.signs.find((s) => this.physics.overlap(this.player,
  s.zone));
  if (sign === undefined) {
    this.messageText.setVisible(false);
  } else {
    this.messageText.setText(sign.message).setVisible(true);
  }
}

```

The message box is one `Text` with a `backgroundColor` and `padding`, fixed to the camera with `setScrollFactor(0)`, and hidden until needed.

Tested with the harness: at the start the box shows the controls; walking right out of the first sign's rectangle hides it; standing by the lava sign shows "Lava ahead - jump it!".

Look for:

- the top-left to centre conversion (the classic bug: the message appears half a sign to the left)
- hiding the box again. The usual first attempt uses `this.physics.add.overlap(player, zone, callback)`, which says when the player is **in** a sign but never when they have **left** - so the message never goes away. The hint points at `physics.overlap` in `update()`; a "was I overlapping last frame?" flag is another good answer

- the message stored with its zone (an interface, or `zone.setData("message", ...)`) rather than a second array kept in step

5. Moving platforms

Project: `solutions/ch11_challenge_5_moving_platforms` (from `ch11_tiled_level`)

Goal: a polyline's points, a chain of tweens, an immovable body - and discovering that Arcade Physics does not carry a rider sideways on a tweened platform.

In Tiled: erase the floating platform over the wide pool, then draw a polyline with Insert Polygon (click the points, then press Enter - or right-click - to finish without closing it), and name it **platform**. The solution's goes across the pool at ground level and then up, so the player can ride it to the higher platform.

`src/scenes/GameScene.ts`

```
// a polyline's points are measured from the object's x and y: turn them  
into world positions  
const points = obj.polyline.map((p) => new Phaser.Math.Vector2((obj.x ?? 0)  
+ p.x, (obj.y ?? 0) + p.y));  
  
const platform = this.add.image(points[0].x, points[0].y,  
PLATFORM_KEY).setScale(PLATFORM_SCALE);  
this.physics.add.existing(platform);  
const body = platform.body as Phaser.Physics.Arcade.Body; // dynamic: made  
by physics.add.existing  
body.setAllowGravity(false); // it floats  
body.setImmovable(true); // the player cannot push it
```

```

// there and back: each point after the first, then back through each point
before the last
const route = [...points.slice(1), ...points.slice(0, -1).reverse()];
let from = points[0];
const steps: Phaser.Types.Tweens.TweenBuilderConfig[] = [];
for (const to of route) {
    const distance = Phaser.Math.Distance.BetweenPoints(from, to);
    steps.push({
        targets: platform,
        x: to.x,
        y: to.y,
        duration: (distance / PLATFORM_SPEED) * 1000,
        completeDelay: PLATFORM_WAIT,
    });
    from = to;
}
this.tweens.chain({ tweens: steps, loop: -1 });

```

Each step's duration comes from its length, so the platform moves at a steady speed. `completeDelay` makes it wait at each point. (`hold` looks tempting, but it only applies to tweens that `yoyo`.)

Arcade's collider lifts the player with the platform, but a tween changes the platform's `x` directly, so its body has no velocity and the player is not carried sideways - the platform slides out from under them. The solution carries the player itself, every frame:

```

private carryPlayer(): void {
    const feet = this.player.body as Phaser.Physics.Arcade.Body;
    for (const platform of this.platforms) {
        const top = platform.image.body as Phaser.Physics.Arcade.Body;
        const standingOn = Math.abs(feet.bottom - top.top) < 2 && feet.right >
top.left && feet.left < top.right;
        if (standingOn) {
            this.player.x = this.player.x + (platform.image.x - platform.lastX);
        }
        platform.lastX = platform.image.x;
    }
}

```

Tested with the harness: over two complete trips the player's position relative to the platform stayed exactly the same (7 pixels left of its centre, 32 above), across, up, down

and back.

Look for:

- the points turned into world positions (forgetting to add `obj.x / obj.y` puts the platform near the top-left corner of the map)
 - a body that ignores gravity and cannot be pushed - otherwise it falls into the pool, or the player pushes it along
 - some way of carrying the player. A good, alternative answer is `body.setDirectControl(true)`, which tells Arcade the body is moved by setting its position and makes it work out a velocity, so the collider carries the rider. In testing it carried the player less reliably than the hand-written version (the rider drifted across the platform over a few trips), which is worth discussing. Moving the platform by velocity instead of tweens (`this.physics.moveTo`) also works, and Arcade then carries the player by itself; it needs more code to know when each point is reached. Chapter 15 comes back to moving platforms
-

6. Spinning coins, as tiles

Project: `solutions/ch11_challenge_6_spinning_coins_as_tiles` (from `ch11_tiled_level`)

Goal: a second tileset and firstgid, Tiled's tile animations, and collecting tiles with an overlap and `removeTileAt` .

In Tiled:

1. File > New > New Tileset: `coin_spin.png` (in `public/assets/spritesheets/`), 32 x 32, **embedded**, named **coins**. Tiled gives it firstgid 17, because `platform_tiles` uses 1-16
2. Edit Tileset, select tile 0, and open the **Tile Animation Editor** (a button on the tileset editor's tool bar). Drag the six tiles into the frame list, each 100 ms
3. a new tile layer, **Pickups**, above Ground; paint coin tile 0 where the coins should be. The editor shows them spinning

`src/scenes/GameScene.ts`

```
// CHALLENGE 6 - the second tileset, "coins", joined to the coin_spin
// picture. Its tiles have GIDs
// from 17 (its firstgid), so they cannot be mixed up with platform_tiles'
// 1-16. Phaser 4 plays
// the tile animation made in Tiled's Tile Animation Editor by itself.
const coinTiles = this.map.addTilesetImage(COINS_TILESET_NAME, COIN_KEY);
if (coinTiles === null) {
    throw new Error(`No tileset "${COINS_TILESET_NAME}" in the map, or no
    picture "${COIN_KEY}" loaded`);
}
this.pickups = this.map.createLayer(PICKUPS_LAYER, coinTiles) as
Phaser.Tilemaps.TilemapLayer;
```

```
// CHALLENGE 6 - an overlap with a tile layer reports EVERY tile under the
// player's body,
// including the empty places (index -1), so only a real coin tile counts
this.physics.add.overlap(this.player, this.pickups, (_player, object) => {
    const tile = object as Phaser.Tilemaps.Tile; // from a tile layer, the
    second object is a Tile
    if (tile.index !== -1) {
        this.collectCoinTile(tile);
    }
});
```

```
// CHALLENGE 6 - take the coin tile out of the layer, and count it
private collectCoinTile(tile: Phaser.Tilemaps.Tile): void {
    this.pickups.removeTileAt(tile.x, tile.y);
}
```

The `coin_spin` sprite sheet, already loaded for the object coins, is used as the tileset's picture as well - one texture can be both. Coin tiles are counted with

```
this.pickups.filterTiles((tile: Phaser.Tilemaps.Tile) => tile.index !== -1) .
```

Tested with the harness: the map has tilesets `platform_tiles` (firstgid 1, 16 tiles) and `coins` (firstgid 17, 6 tiles); `getAnimatedTileId(17, ms)` steps 17, 18, 19, 20 as time passes, and two screenshots 170 ms apart show the coins at different frames; running right along row 16 collects three coin tiles, and the total counts 16 object coins + 10 tiles.

Look for:

- `addTilesetImage` called twice, once per tileset, and the Pickups layer given the right one (or an array of both, if a student paints both tilesets on one layer)
- `tile.index !== -1` in the overlap - without it the first empty place counts as a coin, and the count races up as soon as the game starts
- students who follow Phaser 3 tutorials and write their own tile animation code: Phaser 3 needed a plugin or hand-written code for Tiled's tile animations; **Phaser 4 plays them itself** (both kinds of tilemap layer). A student who animates by hand with a timer and `putTileAt` has done more work than needed, but understood the idea - credit it
- the firstgid question from the hint answered: 17

Slides

Chapter 11

Tilemaps with Tiled

Draw the level. Load the level.



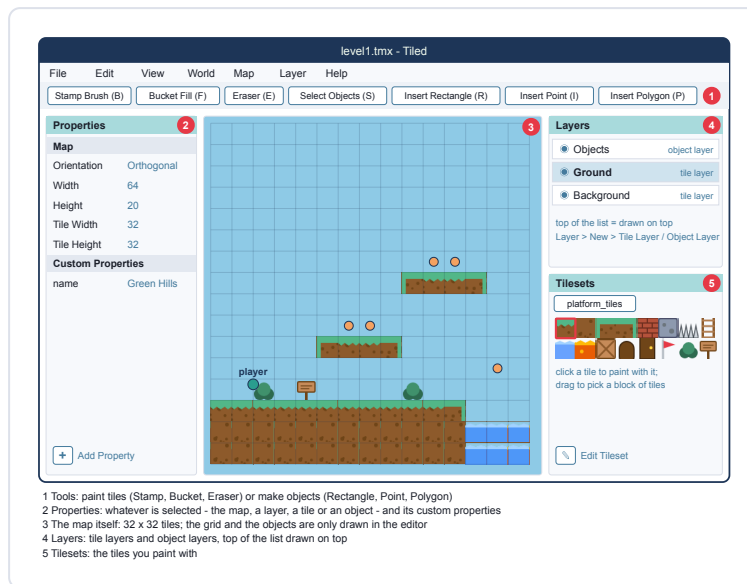
1 / 20

Today

- **Tiled:** tilesets, tile layers, object layers, custom properties
- `.tmx` (source) and `.tmj` (what the game loads)
- GIDs
- `tilemapTiledJSON`, `make.tilemap`, `addTilesetImage`, `createLayer`
- solid tiles **by property**
- reading objects: spawn points, coins, enemies, doors
- the level is **data**, not code

2 / 20

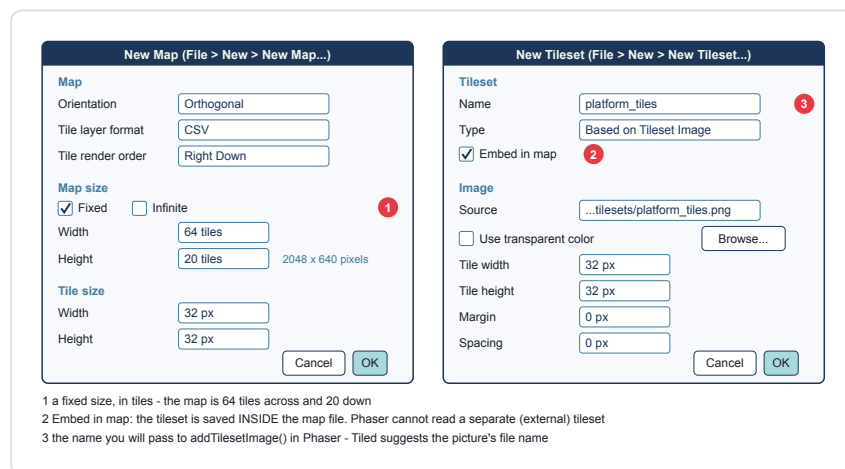
Tiled



Free, mapeditor.org - Windows, macOS, Linux

3 / 20

A new map, a new tileset



Orthogonal · fixed size · CSV · tileset embedded in map

4 / 20

Layers, and painting

- the map starts with one tile layer: rename it **Ground**
- Layer > New > Tile Layer: **Background**, below Ground
- top of the Layers list = drawn on top
- Stamp Brush (B), Bucket Fill Tool (F), Eraser (E)
- **check which layer is selected before you paint**

5 / 20

Custom properties on tiles

The tileset, with a custom property on some of its tiles

0 grass top, 1 dirt, 2 grass left, 3 grass right, 4 brick, 5 stone, 6 spikes, 7 ladder, 8 water, 9 lava, 10 crate, 11 door top, 12 door, 13 flag, 14 bush, 15 sign

☐ collides = true (solid: the player stands on these)
☐ hazard = true (touch one and you start again)

Tiles with no box have no custom properties at all.
The numbers are tile INDEXES within the tileset, from 0 (see the GIDs diagram).

Properties (7 tiles selected)

Tile	ID	Class	Probability
	0		1

Custom Properties

collides ☒ true

1 tick it - the value is true
2 select several tiles first to add it to them all

+ Add Property: name collides, type bool

Edit Tileset → select tiles → + → **collides** , **bool**, ticked

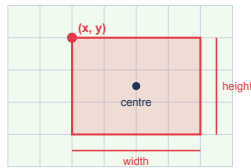
6 / 20

Objects

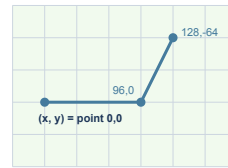
Where Tiled measures each kind of object from



Point: x and y are the point.
A sprite placed there is centred on it.



Rectangle: x, y is the TOP-LEFT corner.
A Zone is placed by its centre:
(x + width / 2, y + height / 2)



Polyline: a list of points,
each measured FROM (x, y).

Layer > New > Object Layer · Insert Point (I) · Insert Rectangle (R) · **Name** them: `player` , `coin` , `enemy` , `goal`

7 / 20

Save, then export

`tiled/level1.tmx`

`public/assets/maps/level1.tmj`

File > Save

- you edit this

File > Export As...

- the game loads this

- after the first time: **File > Export** (Ctrl+E / Cmd+E)
- then **build**, then **refresh**
- "I changed the map and nothing happened" = not exported

8 / 20

Inside a .tmj

level1.tmj - the map

```
"type": "map",
"orientation": "orthogonal",
"width": 64, "height": 20,
"tilewidth": 32,
"tileheight": 32,
"infinite": false,
"properties": [ ... ],
"tilesets": [ ... ],
"layers": [ ... ],
"nextlayerid": 4,
"nextobjectid": 21,
"version": "1.10"
```

Keys are written in alphabetical order;
"..." marks what has been left out here.

Phaser reads all of it with
`this.load.tilemapTiledJSON(key, file)`
and builds the map with
`this.make.tilemap({ key })`

tilesets[0] - embedded in the map

```
"name": "platform_tiles",      <- addTilesetImage("platform_tiles", ...)
"firstgid": 1,
"image": "../tilesets/platform_tiles.png",
"columns": 8, "tilecount": 16, "tilewidth": 32, ...
"tiles": [ { "id": 0, "properties": [
  { "name": "collides", "type": "bool", "value": true } ] }, ... ]
```

layers[0] and [1] - tile layers

```
"name": "Ground", "type": "tilelayer",      <- createLayer("Ground", ...)
"width": 64, "height": 20,
"data": [ 0, 0, 0, ... ]                  one GID per tile, row by row:
      1, 1, 1, 1, 4, 0, 0, 0, 3, ... ] 64 x 20 = 1280 numbers
```

layers[2] - the object layer

```
"name": "Objects", "type": "objectgroup", <- getObjectLayer("Objects")
"objects": [
  { "id": 1, "name": "player", "point": true, "x": 64, "y": 520, ... },
  { "id": 2, "name": "coin", "point": true, "x": 200, "y": 432, ... },
  { "id": 18, "name": "enemy", "point": true, "x": 1232, "y": 520,
    "properties": [ { "name": "speed", "type": "float", "value": 60 } ] },
  { "id": 20, "name": "goal", "x": 1952, "y": 480,
    "width": 32, "height": 64, ... }
]
```

9 / 20

GIDs

Tile index (in the tileset) and GID (in the map)

tileset												
index	0	1	2	3	4	5	6	7	8	9	10	11
GID	1	2	3	4	5	6	7	8	9	10	11	12

GID = firstgid + index. This tileset's firstgid is 1, so GID 0 is left free to mean "no tile".
A second tileset in the same map would start where this one ends - firstgid 17.

row 17 of the Ground layer, columns 7 to 16, as saved in the .tmj:

drawn												
tile.index	1	1	1	1	4	-1	-1	-1	3	1		

In Phaser a Tiled tile's index IS its GID - grass top is 1, not 0 - and an empty place is -1.
So `setCollision(1, 2, 3)` means GIDs. Choosing tiles by property avoids counting at all.

10 / 20

Loading the map

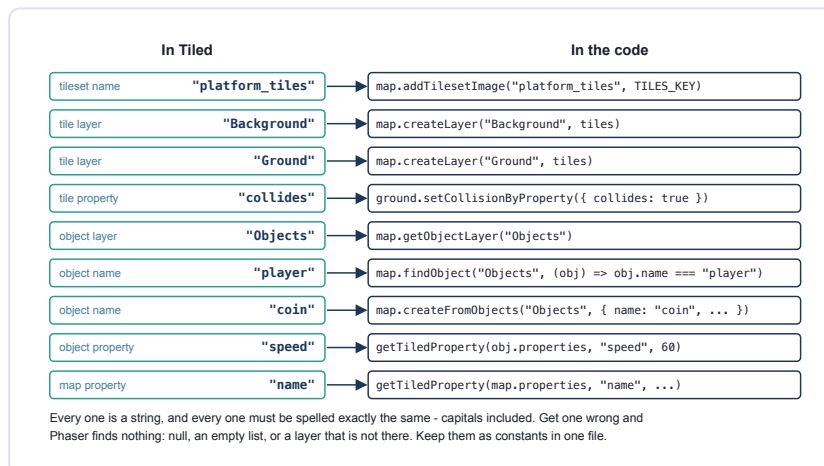
```
// preload()
this.load.tilemapTiledJSON(MAP_KEY, MAP_FILE); // the .tmj
this.load.image(TILES_KEY, TILES_FILE);       // its picture - separately

// create()
this.map = this.make.tilemap({ key: MAP_KEY });
const tiles = this.map.addTilesetImage(TILESET_NAME, TILES_KEY);
this.map.createLayer(BACKGROUND_LAYER, tiles);
this.ground = this.map.createLayer(GROUND_LAYER, tiles) as Phaser.Tilemaps.TilemapLayer;
this.ground.setCollisionByProperty({ collides: true });
```

addTilesetImage(name in Tiled, key in Phaser)

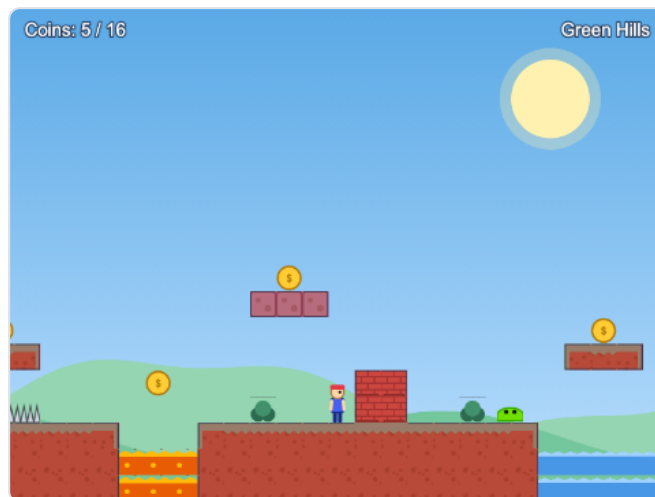
11 / 20

The names must match



12 / 20

Press D



```
tiles.some((tile) => tile.properties.hazard === true); // any tile property
```

13 / 20

Three ways into an object layer

```
// one object
const spawn = this.map.findObject(OBJECTS_LAYER, (obj) => obj.name === "player");

// a Sprite for every match
const coinSprites = this.map.createFromObjects(OBJECTS_LAYER, { name: "coin", key: COIN_KEY });

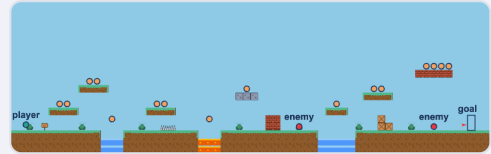
// all of them, to do as you like with
for (const obj of objectLayer.objects) {
  if (obj.name === "enemy") {
    const speed = getTiledProperty(obj.properties, "speed", DEFAULT_SLIME_SPEED);
```

Object and map properties: a list of { name, type, value }

14 / 20

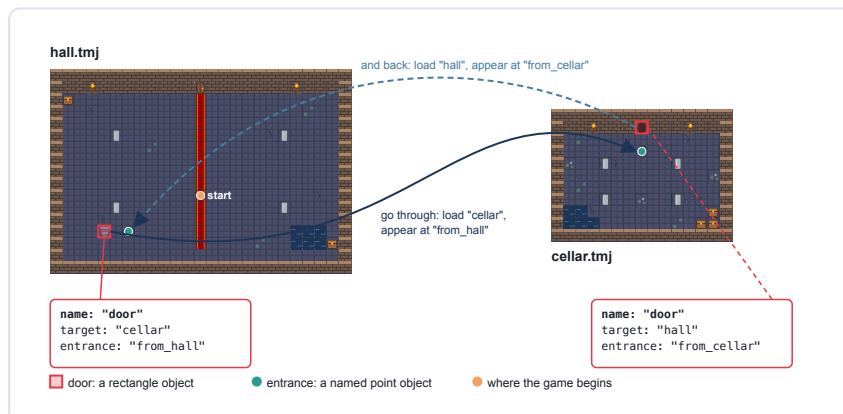
Try it: Tiled Level

- build, serve, play `ch11_tiled_level`
- open `tiled/level1.tmx` in Tiled
- change a slime's `speed` ; add a platform and two coins
- export, build, refresh



15 / 20

Doors between maps



16 / 20

One scene, restarted

```
init(data: Partial<RoomData>): void {
  this.room = {
    map: data.map ?? FIRST_ROOM.map,
    entrance: data.entrance ?? FIRST_ROOM.entrance,
  };
  this.leaving = false;
}
// preload(): only this room's map - skipped if already in the cache
this.load.tilemapTiledJSON(this.room.map, MAPS_FOLDER + this.room.map + MAP_EXTENSION);

// goThrough(door), after the fade:
const next: RoomData = { map: door.target, entrance: door.entrance };
this.scene.restart(next);
```

17 / 20

Try it: Tiled Doors

- down the stairs; back through the cellar door
- where is the list of rooms? (there isn't one)
- why are the entrance points away from the doors?



18 / 20

Summary

- Tiled: tilesets, tile layers, object layers, custom properties
- edit `.tmx` , **export** `.tmj` ; embedded tileset, fixed size, CSV
- GID = firstgid + index; in Phaser, `tile.index` is the GID, empty = -1
- `tilemapTiledJSON` → `make.tilemap({ key })` → `addTilesetImage` → `createLayer`
- `setCollisionByProperty({ collides: true })` ; `tile.properties`
- `findObject` , `filterObjects` , `createFromObjects` , `getObjectLayer(...).objects`
- names are the contract: constants, spelled exactly

19 / 20

Challenges

1. **Above the player** - a layer drawn over the player
2. **Gems** - coins with a `value` property
3. **A third room** - behind the closed door (how much code?)
4. **Signposts** - rectangles with messages
5. **Moving platforms** - along a polyline
6. **Spinning coins, as tiles** - a second tileset, animated in Tiled

Next: **Chapter 12 - Higher or lower**

20 / 20